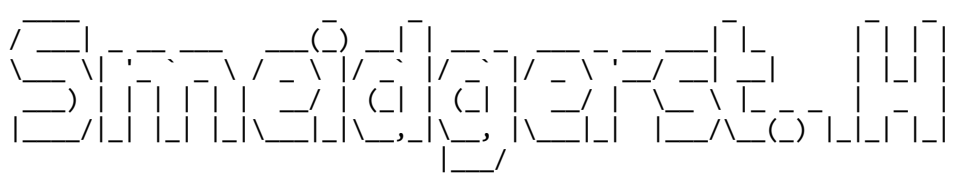
Time Complexity & Big O Notation for Algorithms ⏰💻

Understanding Algorithm Efficiency in Data Structures and Algorithms



1. Introduction to Time Complexity ⏱️

- Time complexity is a fundamental concept in Computer Science, especially in Data Structures and Algorithms (DSA).
- It is crucial for evaluating and comparing the efficiency of different algorithms.
- Understanding time complexity is vital for interviews and competitive programming.

2. What is Time Complexity?

Time complexity is defined as:

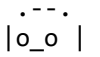
- ***The time taken by an algorithm as a function of the length of its input.***

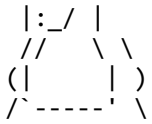
This means as the input size increases, how does the algorithm's performance change?

3. Why is Time Complexity Important? ⚖️

- **Algorithm Comparison:** Its primary use is to *compare two or more algorithms* solving the same problem to determine which one is better.
- **Objective Measurement:** It provides a machine-independent way to evaluate algorithm performance, unlike directly running and timing code, which can vary based on hardware or other processes.
- **Example: Sum from 1 to N**
  - **Method 1: Formula** `N * (N + 1) / 2`
  - **Method 2: Loop** Iterate from 1 to N, adding each number.
  - Time complexity helps objectively determine which method is more efficient, regardless of the machine it runs on.

4. Big O Notation 📈





- **Purpose:** The Big O Notation is used to *quantify the time complexity* of an algorithm.
- **Worst-Case Scenario:** It describes the *upper bound* of an algorithm's running time, representing the worst-case scenario. This means an algorithm will take *at most* this much time, though it might take less.
- **Example: Linear Search**
  - Searching for an element (key) in an array.
  - **Worst Case:** The key is either the last element or not present in the array. In this case, the algorithm must traverse the entire array.
  - Big O focuses on this worst-case performance.

4.1 Rules for Calculating Big O Notation

When determining Big O complexity, follow these rules:

- **Ignore Smaller Powers:** Only consider the term with the *highest power* of 'n' (input size).
  - Example: For `3n^2 + 5n`, `n^2` is the dominant term.
- **Ignore Constants:** Discard constant factors associated with the highest power term.
  - Example: For `3n^2`, ignore `3`. The complexity is `O(n^2)`.

4.2 Big O Notation Examples

- **Example 1:** `3n^2 + 5n`
  - Highest power: `n^2`
  - Ignore constant: `3`
  - **Time Complexity:** `O(n^2)`
- **Example 2:** `n + 100 log n`
  - Compare growth: `n` grows faster than `100 log n`.
  - **Time Complexity:** `O(n)`
- **Example 3:** `3n^3 + 4n^5`
  - Highest power: `n^5`
  - Ignore constant: `4`
  - **Time Complexity:** `O(n^5)`
- **Example 4:** `10000` (a constant value, not dependent on 'n')
  - No 'n' term, so it's a constant time operation.
  - **Time Complexity:** `O(1)`
- **Other `O(n)` Examples:** `n/4`, `n + 100/4`, `5n + 3 log n` (`5n` dominates).
- **Other `O(n^2)` Examples:** Any term with `n^2` as the highest power, even if divided by a constant (e.g., `n^2/5` is still `O(n^2)`).

5. Other Asymptotic Notations (Briefly)

- **Theta (Θ) Notation:** Defines the *exact bound* (both upper and lower) of an algorithm's running time. It represents the average-case complexity.

- **Omega (Ω) Notation:** Defines the *lower bound* of an algorithm's running time, representing the best-case scenario.
- **Focus:** In Computer Science, Big O Notation is predominantly used because it provides a safe upper limit, ensuring an algorithm will perform no worse than a certain bound.

6. Time Complexity of Recursive Functions 🌳

Calculating time complexity for recursive functions can be challenging, but the *Tree Method* simplifies this.

6.1 Tree Method Steps

1. **Convert to Recurrence Relation (T Function):** Represent the recursive function's time complexity as a `T(n)` function.
    - `T(n)` represents the time taken for an input of size `n`.
    - Break down the function into:
        - Recursive calls (e.g., `2T(n/2)` for two calls on half input).
        - Non-recursive work (e.g., loops, constant operations).
2. **Draw the Recursion Tree:**
    - The root represents the initial call and its non-recursive work.
    - Children represent the recursive calls and their non-recursive work.
    - Continue expanding until a base case is reached.
3. **Calculate Work at Each Level:** Sum the non-recursive operations performed at each level of the tree.
4. **Determine Tree Height:** Find out how many levels the tree has.
5. **Total Work:** Multiply the work per level by the height of the tree.
6. **Final Big O:** Simplify the total work into Big O notation (ignoring constants).

6.2 Recursive Function Example Walkthrough

Consider a function: `fun(n)`

```
function fun(n) {
    if (n == 0) return; // Base case
    for (i = 0 to n) { // Loop performing 'n' operations
        // Some constant operations (c)
    }
    fun(n/2); // Recursive call 1
    fun(n/2); // Recursive call 2
}
```

1. **Recurrence Relation:**
    - Two recursive calls: `2 * T(n/2)`
    - For loop: performs `n` operations. Let's say each operation costs `c`. So, `nc`.

    - Constant operations: some constant time, often absorbed into the `nc` term or represented as `c`.
    - So, `T(n) = 2T(n/2) + nc`
2. **Recursion Tree:**
    - **Level 0:** Root node: `nc` (non-recursive work for `fun(n)`)
    - **Level 1:** Two child nodes, each for `fun(n/2)`.
        - Non-recursive work for each: `(n/2)c`
        - Total work at Level 1: `(n/2)c + (n/2)c = nc`
    - **Level 2:** Four child nodes, each for `fun(n/4)`.
        - Non-recursive work for each: `(n/4)c`
        - Total work at Level 2: `4 * (n/4)c = nc`
    - *Observation:* At every level, the total non-recursive work done is `nc`.
3. **Tree Height:**
    - The input size halves at each step: `n, n/2, n/4, ..., 1` (or until it reaches the base case, typically 0 or 1).
    - The number of times you can divide `n` by 2 until it reaches 1 is $\log_2 n$.
    - So, the height of the tree is `log n`.
4. **Total Work & Big O:**
    - Total work = (Work per level) * (Number of levels)
    - Total work = `nc * log n`
    - Ignoring the constant `c`: `O(n log n)`
    - **Time Complexity:** `O(n log n)`

6.3 Practice Questions

Try to find the time complexity of these recursive functions:

1. `T(n) = T(n/4) + T(n/2) + c`
2. `T(n) = 2T(n-1) + c`

(Answers can be found on the instructor's Telegram channel.)

7. Online Judges and the 10^8 Operations Rule 💻📊

When participating in competitive programming or solving problems on online platforms like Codeforces, you often see *constraints* on the input size (e.g., `N < 10^5` or `N < 100`). These constraints are critical for determining which algorithm to use.

- **The Rule:** Online judges typically allow a maximum of approximately ***`10^8` operations per second***. If your code performs significantly more operations than this within the given time limit (usually 1-2 seconds), it will result in a "Time Limit Exceeded" (TLE) error.
- **Using Constraints to Choose Algorithms:**
    - **N is small (10-11):** Algorithms with very high complexity like `O(N!)` (factorial) or `O(N^6)` might be acceptable.
        - Example: `11!` is approximately `3.99 * 10^7`, which is less than `10^8`.
    - **N between 15-18:** Algorithms like `O(2^N)` (exponential) might be acceptable.
    - **N = 100:** You can typically go up to `O(N^4)`.
        - Example: `100^4 = 10^8` operations.
    - **N = 400:** `O(N^3)` might be acceptable.

        - Example: `400^3 = (4 * 10^2)^3 = 64 * 10^6 = 6.4 * 10^7` operations.
    - **N = 10^5:** You'll likely need `O(N log N)` or `O(N)` algorithms.
        - Example: For `N = 10^5`, `N log N` is approx. `10^5 * log(10^5)` which is roughly `10^5 * 17 = 1.7 * 10^6` operations.
    - **N = 10^6:** Usually requires `O(N)` or `O(log N)`.
- **Strategy:** When you see the constraints for `N`, immediately think about the highest possible time complexity (e.g., `O(N^2)`, `O(N log N)`) that would fit within the `10^8` operations rule. This helps in selecting the appropriate algorithm. For example, if `N = 10^5` and you determine `O(N log N)` is the max, consider algorithms involving sorting (often `N log N`) or data structures that offer `log N` operations.

8. Next Steps & Practice 🧠

- Continue solving problems related to time complexity.
- Recommended resource: Interview Bit has a dedicated section for time complexity questions (MCQs and problem-solving).
- Practice applying Big O notation rules to various functions and algorithms.